

Concurrency

Module – 4

Lecture 1

Interprocess Communication

- Processes executing concurrently in the operating system may be either independent processes or cooperating processes.
- Reasons for providing an environment that allows process cooperation: Information sharing, Computation speedup and Modularity

Interprocess Communication

- Cooperating processes require an **interprocess communication (IPC)** mechanism that will allow them to exchange data.
- Two mechanisms: **Shared Memory** and **Message passing**

Shared Memory and Message passing

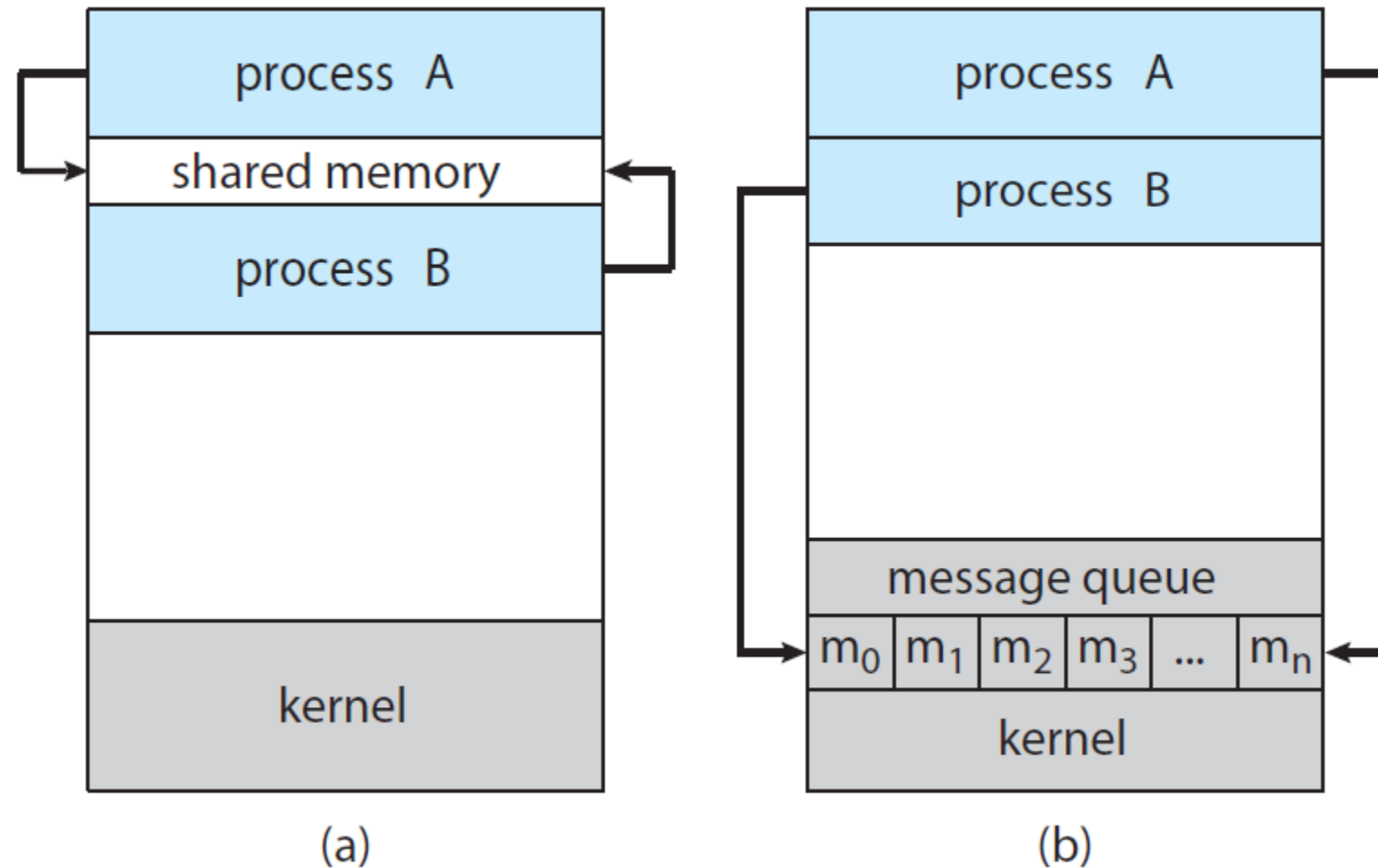


Figure 3.11
Communications models.
(a) Shared memory.
(b) Message passing

Shared Memory and Message passing

- Message passing is useful for exchanging smaller amounts of data, because no conflicts need be avoided.
- Message passing is also easier to implement in a distributed system than shared memory.
- Shared memory can be faster than message passing (More System calls are used by message passing)

IPC in Shared-Memory Systems

- IPC using shared memory requires communicating processes to establish a region of shared memory.
- It resides in the address space of the processes
- Processes can exchange information by reading and writing data in the shared areas.
- The form of the data and the location are determined by these processes and are not under the operating system's control

IPC in Shared-Memory Systems

- Example for cooperating process: *Producer – Consumer Problem*
- A **producer** process produces information that is consumed by a **consumer** process
- Example: Compiler – Assembler – Loader
- One solution to the Producer-Consumer problem uses shared memory.
- A buffer will reside in a region of memory that is shared by the producer and consumer processes

IPC in Shared-Memory Systems

- A producer can produce one item while the consumer is consuming another item.
- The producer and consumer must be **synchronized**, so that the consumer does not try to consume an item that has not yet been produced.
- **unbounded buffer and bounded buffer**

Memory shared by Producer and consumer Processes

```
#define BUFFER_SIZE 10

typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

The producer process using shared memory

```
item next_produced;

while (true) {
    /* produce an item in next_produced */

    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */

    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

The consumer process using shared memory

```
item next_consumed;

while (true) {
    while (in == out)
        ; /* do nothing */

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next_consumed */
}
```

Producer – Consumer Problem

- **Write a C program to implement the producer consumer problem.**

IPC in Message-Passing Systems

- Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.
- useful in a distributed environment

Example: an Internet chat program

IPC in Message-Passing Systems

- A message-passing facility provides at least two operations:
send(message) and receive(message)
- Messages sent by a process can be either fixed or variable in size
- If processes P and Q want to communicate, they must send messages to and receive messages from each other: a **communication link** must exist between them.

IPC in Message-Passing Systems

Methods for logically implementing a link for `send()/receive()` operations:

1. Direct or indirect communication
2. Synchronous or asynchronous communication
3. Automatic or explicit buffering

IPC in Message-Passing Systems

Direct communication

- Here, each process that wants to communicate must explicitly name the recipient or sender of the communication
 - `send(P, message)`—Send a message to process P.
 - `receive(Q, message)`—Receive a message from process Q.
- A link is established automatically between every pair of processes that want to communicate.

IPC in Message-Passing Systems

symmetry addressing and a *symmetry* addressing

Example for a *symmetry* in addressing

- `send(P, message)`—Send a message to process P.
- `receive(id, message)`—Receive a message from any process

IPC in Message-Passing Systems

- With indirect communication, the messages are sent to and received from **mailboxes, or ports**.
- A **mailbox** can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed.
- Each mailbox has a **unique identification**.

IPC in Message-Passing Systems

- A process can communicate with another process via a number of different mailboxes, but two processes can communicate only if they have a **shared mailbox**.
- Primitives for indirect communication
 - `send(A, message)`—Send a message to mailbox A.
 - `receive(A, message)`—Receive a message from mailbox A.

IPC in Message-Passing Systems

- A process can communicate with another process via a number of different mailboxes, but two processes can communicate only if they have a **shared mailbox**.
- A mailbox may be **owned** either by a process or by the operating system
- Primitives for indirect communication
 - `send(A, message)`—Send a message to mailbox A.
 - `receive(A, message)`—Receive a message from mailbox A.

IPC in Message-Passing Systems

- Message passing may be either **blocking** or **nonblocking**— also known as **synchronous** and **asynchronous**.
 - **Blocking send.** The sending process is blocked until the message is received by the receiving process or by the mailbox.
 - **Nonblocking send.** The sending process sends the message and resumes operation.
 - **Blocking receive.** The receiver blocks until a message is available.
 - **Nonblocking receive.** The receiver retrieves either a valid message or a null.

IPC in Message-Passing Systems

Buffering

- Whether communication is direct or indirect, messages exchanged by communicating processes reside in a temporary queue.
- Basically, such queues can be implemented in three ways:
Zero capacity, Bounded capacity and Unbounded capacity

IPC in Message-Passing Systems

Buffering

- **Zero capacity.** The queue has a maximum length of zero
- **Bounded capacity.** The queue has finite length n ; thus, atmost n messages can reside in it.
- **Unbounded capacity.** The queue's length is potentially infinite

The zero-capacity case is sometimes referred to as a **message system with no buffering**. The other cases are referred to as systems with **automatic buffering**